

Reviewer Pack — Minimal Order of Noncommutative Crossed Products and Communi...

Entry 1da457004711 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-02 16:28:04 UTC

Minimal Order of Noncommutative Crossed Products and Communication Complexity Rank

Entry ID: 1da457004711 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-02 16:28:04 UTC

1. Conjecture

Field A (mathematical branch): Noncommutative Geometry (Crossed Products) **Field B** (complexity object): Communication Complexity

Statement:

For every n -ary communication protocol P , the minimal order of a noncommutative crossed product associated with the algebraic structure of P is linearly correlated with its communication complexity rank $r(P)$, such that the minimal order $= \Theta(r(P))$.

Rationale (proposer's reasoning):

Noncommutative crossed products have been used to study the structure of algebras and their representations, which could potentially provide a new perspective on the complexity of communication protocols. This conjecture aims to explore whether the algebraic properties of noncommutative crossed products can be leveraged to understand the complexity of communication tasks.

Taxonomy category: CommunicationComplexity (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): da0504a5a545c07a

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the Pearson correlation coefficient from the data set of n -ary protocols exceeds 0.7, with each protocol having at least 100 seeds. The conjecture is falsified if the correlation coefficient is less than or equal to 0.3.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - 'noncommutative geometry crossed products' AND 'communication complexity rank' - 'minimal order noncommutative crossed product' related TO 'communication complexity' - crossed products in noncommutative geometry impact on communication complexity

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=0, elapsed=0.0s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_communication_protocol(n):
        protocol = []
        for _ in range(n):
            protocol.append(random.randint(1, 10))
        return protocol

    def communication_complexity_rank(protocol):
        rank = 0
        for i in range(len(protocol)):
            for j in range(i + 1, len(protocol)):
                if protocol[i] != protocol[j]:
                    rank += 1
        return rank

    def noncommutative_crossed_product_order(protocol):
        order = sum(1 for x in protocol if x > 5)
        return order

    n_values = [5, 10, 15, 20, 30, 40]
    results = []

    for n in n_values:
        protocol = generate_communication_protocol(n)
        rank = communication_complexity_rank(protocol)
        order = noncommutative_crossed_product_order(protocol)
```

```

        results.append({
            "n": n,
            "protocol": protocol,
            "rank": rank,
            "order": order
        })

    if len(results) < 30:
        return {
            "metric_name": "communication_complexity_rank",
            "metric_value": None,
            "instances_tested": len(results),
            "n_max": max(result["n"] for result in results),
            "conjecture_holds": False,
            "counterexample": "not_enough_instances"
        }

    ranks = [result["rank"] for result in results]
    orders = [result["order"] for result in results]

    mean_rank = sum(ranks) / len(ranks)
    mean_order = sum(orders) / len(orders)

    covariance = sum((r - mean_rank) * (o - mean_order) for r, o in zip(ranks, orders))
    variance_rank = sum((r - mean_rank) ** 2 for r in ranks)
    variance_order = sum((o - mean_order) ** 2 for o in orders)

    correlation_coefficient = covariance / math.sqrt(variance_rank * variance_order)

    return {
        "metric_name": "communication_complexity_rank",
        "metric_value": correlation_coefficient,
        "instances_tested": len(results),
        "n_max": max(result["n"] for result in results),
        "conjecture_holds": 0.3 < correlation_coefficient <= 0.7,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(arg) for arg in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []

    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    if all(result["conjecture_holds"] for result in results):
        support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)
        print(f"RESULT: SUPPORTED mean={sum(result['metric_value'] for result in results) / len(results)}")
    elif any(not result["conjecture_holds"] for result in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"correlation_coefficient_outside_threshold\" first_failing_seed={first_failing_seed}")

```

```

else:
    print("RESULT: INCONCLUSIVE no_trials_supported")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

, 'conjecture_holds': False, 'counterexample': 'not_enough_instances'}
TRIAL: {'metric_name': 'communication_complexity_rank', 'metric_value': None, 'instances_tested': 6,
TRIAL: {'metric_name': 'communication_complexity_rank', 'metric_value': None, 'instances_tested': 6,
TRIAL: {'metric_name': 'communication_complexity_rank', 'metric_value': None, 'instances_tested': 6,
TRIAL: {'metric_name': 'communication_complexity_rank', 'metric_value': None, 'instances_tested': 6,
TRIAL: {'metric_name': 'communication_complexity_rank', 'metric_value': None, 'instances_tested': 6,
TRIAL: {'metric_name': 'communication_complexity_rank', 'metric_value': None, 'instances_tested': 6,
TRIAL: {'metric_name': 'communication_complexity_rank', 'metric_value': None, 'instances_tested': 6,
TRIAL: {'metric_name': 'communication_complexity_rank', 'metric_value': None, 'instances_tested': 6,
TRIAL: {'metric_name': 'communication_complexity_rank', 'metric_value': None, 'instances_tested': 6,
TRIAL: {'metric_name': 'communication_complexity_rank', 'metric_value': None, 'instances_tested': 6,
TRIAL: {'metric_name': 'communication_complexity_rank', 'metric_value': None, 'instances_tested': 6,
RESULT: FALSIFIED counterexample="correlation_coefficient_outside_threshold" first_failing_seed=11

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

The test only considers a small range of n values (5 to 40) and does not scale with n, which is insufficient for confirming the conjecture’s validity over all possible communication protocols. The metric used to measure ‘order’ is trivially dependent on the value 5 in the noncommutative_crossed_product_order function, making it unsuitable for a meaningful comparison.

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Safety rail: critic_challenge_falsified | original: The correlation coefficient for the tested data set was outside the threshold for supporting the conjecture. Additionally, the critic challenged the v | next: Further investigation is needed to determine a more comprehensive and reliable method for measuring the order of noncommutative crossed products and their correlation with communication complexity rank. This may involve expanding the range of n values tested and revisiting the definition

11. Audit log (LLM calls)

Total LLM calls: 14

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13523
2	propose	ollama_remote	glm4:latest	0	0	12861
3	propose	ollama_remote	glm4:latest	0	0	13539
4	propose	ollama_remote	glm4:latest	0	0	24022
5	propose	ollama_remote	glm4:latest	0	0	14273

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
6	preregistration	ollama_remote	glm4:latest	0	0	9465
7	novelty	ollama_remote	glm4:latest	0	0	13880
8	novelty	ollama_remote	glm4:latest	0	0	9690
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14333
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15558
11	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8684
12	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13459
13	critic	ollama_remote	glm4:latest	0	0	15962
14	judge	ollama_remote	glm4:latest	0	0	16676

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 195926 ms total latency. Provider mix: {'ollama_remote': 14}

(full prompt+response transcripts available in `research/audit/1da457004711.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/1da457004711.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/1da457004711.tar.gz` (if generated)